



INSTALLATION PLANNING GUIDE FOR K2VIEW FABRIC SERVICES ON MICROSOFT AZURE

K2view Data Product Platform Services

Version: 1.5

Date: 03 November 2025

This document contains copyrighted work and proprietary information belonging to K2view. All intellectual property rights (including, without limitation, copyrights, trade secrets, trademarks, etc.) evidenced by or embodied in and/or attached, connected, or related to this document, as well as any information contained herein, are and shall be owned solely by K2view. K2view does not convey to you an interest in or to this document, the information contained herein, or its intellectual property rights, but only a personal, limited, fully revocable right to use the document solely for review purposes. Unless explicitly set forth otherwise, you may not reproduce by any means any document and/or copyright contained herein.

Information in this document is subject to change without notice. Corporate and individual names and data used in the examples herein are fictitious unless otherwise noted.

Copyright © 2025 K2view Ltd. / K2VIEW LLC. All rights reserved.

The following are trademarks of K2view: K2view logo, K2view's platform.

K2view reserves the right to update this list document from time to time.

Overview

This document is a planning guide for installing K2view Fabric Services on Microsoft Azure. It details all necessary steps, prerequisites, and configurations to ensure a successful deployment and is designed for IT professionals, system administrators, and project owners who are involved in implementing K2view Fabric's Data Product Platform on Azure infrastructure.

The guide outlines the process, from initial planning and prerequisite validation to provisioning resources, configuring security settings, and setting up container registries and TLS certificates. It includes a checklist and worksheet to capture essential information required at each stage, making the process more manageable and organized.

Following this guide will help you prepare for deploying K2view Fabric Services.

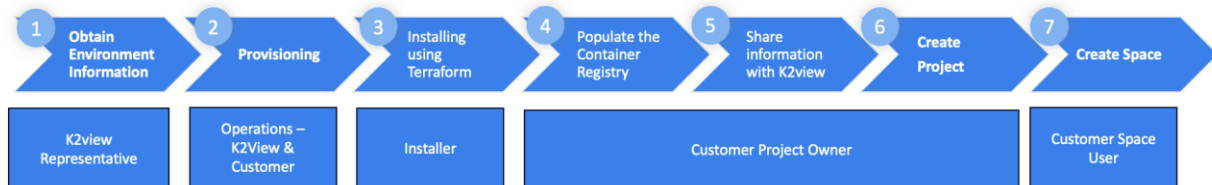
Table of Contents

Overview.....	2
1. Planning and Installation Step Overview	4
Step 1 – Obtaining Environment Information	4
Step 2 – Provisioning	4
Step 3 – Install the K2view Fabric Services.....	5
Step 4 – Populate the Container Registry with K2view Docker images.....	5
Step 5 – Share Container Registry and Domain Name information with K2view	5
Step 6 – Create the K2view Project	5
Step 7 - Create a K2view Space	5
2. Provisioning	6
2.1. Information Required by K2view – Before the Installation.....	6
2.2. K2view Provisioning	6
2.3. Information Required by K2view – Before Proceeding to Step 5	7
3. Constraints.....	7
4. Prerequisites	7
4.1. Internet Access	7
4.2. Whitelisting	8
4.3. DNS	8
4.4. TLS keys and certificates	8
4.5. Permissions and Roles	9
4.6. Persistent Storage	9
4.7. Tools.....	10
4.8. Git Code Repository	10
4.9. Container Registry.....	10
5. Installation Checklist Worksheet.....	12
6. Instructions Summary.....	16
6.1. Cloning the K2view Blueprints	16
6.2. Configuring Your Deployment.....	16
6.3. Populating Your Azure Container Registry	17
6.3.1. Commands to Pull Containers from the K2view container registry	17
6.3.2. Commands to Push Containers to the Azure Container Registry	18
6.3.3. Selecting a Different Azure Container Registry or OCI-Compliant Registry	20
6.4. Sign In to K2cloud.....	21
6.5. Create the Project	22
6.6. Creating Your First Fabric Space	25
7. Context-Based URLs	27

1. Planning and Installation Step Overview

Planning and capturing the necessary information before installing K2view Fabric Services on Microsoft Azure will help ensure a smooth installation experience. The customer and K2view will gather and share information used to provision the environment.

The Installation steps



Step 1 – Obtaining Environment Information

A K2view representative will discuss the prerequisites to complete and review the [Prerequisites](#) section and the [Installation Checklist Worksheet](#).

This review should be used to determine the components to be provisioned based on your use case.

Step 2 – Provisioning

K2view Provisioning

K2view will create a K2view container registry account, user accounts, and associated configurations for you. Some of the information needed to carry out this function will be obtained during Step 1. Other information K2view needs to complete your environment's configuration will need to be provided by you during Step 5.

Customer Preparation

You will need to gather the information described in the [Prerequisites](#) section. You can use the [Installation Checklist Worksheet](#) to capture this information. This includes Git details, obtaining a TLS certificate (PEM encoded) and its private key (PEM encoded), and the names of entities. The certificate and private key files need to be base64 encoded. You must also review what is required to perform the installation. These are listed in the [Prerequisites](#) section.

Step 3 – Install the K2view Fabric Services

Privileges

The installation will be run by someone with the necessary [roles and privileges](#) within your Azure Subscription.

Blueprints

The installation requires configuring the AKS Terraform configuration you generated using the K2view Terraform Blueprints. To obtain the blueprint, follow the instructions in the [Cloning the K2view Blueprints](#) section.

Configuring your Terraform and Performing the Installation

The [Configuring Your Deployment](#) section describes the choices you can make to configure your environment. K2view can assist you in configuring your environment.

Step 4 – Populate the Container Registry with K2view Docker images

The K2view project owner will load the selected Container Registry with K2view Docker images identified by your K2view representative during Step 1. Refer to the [Populating Your Azure Container Registry](#) section for instructions.

Step 5 – Share Container Registry and Domain Name information with K2view

The customer project owner must share the following information to complete the configuration for the tenant's creation of Projects and Spaces.

- The fully qualified domain name (FQDN) used for your Domain Ingress
- The list of Container Registry locations of your Docker images

This information may have been identified during pre-planning, but the specific information should be confirmed to avoid misconfiguration. We recommend that you track this information using the [Installation Checklist Worksheet](#).

Step 6 – Create the K2view Project

To create the K2view Project, refer to the [Creating Your Fabric Project](#) section.

Step 7 - Create a K2view Space

To create your K2view Space refer to the [Creating Your First Fabric Space](#) section.

2. Provisioning

2.1. Information Required by K2view – Before the Installation

Before the installation. K2view needs information to provision your environment.

- **First Admin user:** the user that will set up the K2view project and create the first space. Additional user accounts can also be created.

The information required for users includes:

- Name
 - Email address
 - Phone number with country code included
-
- **A site name and purpose** (could be something like POT, DEV, NPRD, PROD). This information will be used by K2view to better differentiate your first site from others you might create in the future.
 - **The domain name of your Kubernetes Ingress Controller.** If not available before the installation, it can be provided before proceeding to Step 5. You will need this domain name to create your Ingress Controller, register a DNS A record, and create a certificate to secure access to your K2view spaces.

2.2. K2view Provisioning

K2view will provision several things for you:

- Your tenant and its configurations
- Your initial K2view Admin user
- A K2view container registry account for you to access K2view images
- A K2view Cloud Mailbox ID for you to connect to the K2cloud Orchestrator service

K2view will also provide you with additional information, such as:

- In consultation with your K2view representative, Docker images and their associated versions will be recommended. You will need this information to use docker pull commands to obtain images such as fabric, fabric-studio, and the k2-cloud-deployer images.
- K2view may also provide you access to k2exchange.k2view.com, depending on your needs and those identified by your K2view representative.

2.3. Information Required by K2view – Before Proceeding to Step 5

Before you proceed to Step 5, you must pause to provide K2view with information that will allow it to finalize the configuration of your environment. K2View needs:

- The location of the images you loaded to your container registry from the K2view container registry.
 - For example, if you pushed fabric-studio:8.2.1_40 to your container registry, you will need to provide K2view with its location value in the form of [yourRegistry]/k2view/fabric-studio:8.2.1_40 (or the location you configured)
- We need to know the specific images (e.g. fabric-studio) and versions (e.g., 8.2.1_40) you used. A K2view representative will recommend these.
- When configuring your Kubernetes nodes, K2view needs the [“Zero Redundant Storage”](#) type of storage to configure your environment. Refer to the [Persistent Storage](#) section of the [Prerequisites](#) section for details.

3. Constraints

You must know some constraints when installing the K2view Fabric Services within your Azure subscription.

- **Azure Resource Registration.** You must have at least a **contributor** role with permission to manage AKS resources, virtual networks, managed identities, role assignments, and the ability to provision necessary computing and resources.
- **Region and availability zones.** Please note that the installation requires you to use an Azure region with multiple availability zones. Not all Azure Regions support multiple availability zones.

4. Prerequisites

Specific prerequisites must be met, and planning steps must be taken to install K2view Fabric Services, set up the Container Registry, and create a K2view Project and a Space.

4.1. Internet Access

K2view Software

1. The installation presumes that you have Internet access from which to obtain Fabric images from the K2view container registry and perform a Git clone to your machine
2. You need a K2view container registry account to obtain a Fabric Studio docker image. Your K2view account representative can arrange this for you. It is required as part of Step 4 of the installation.

Internet Access Required

Internet access is required to perform this installation. You will need access to:

1. Github.com to clone K2view’s blueprints at <https://github.com/k2view/blueprints.git>
2. K2view’s container registry Docker Image repository at <https://docker.share.cloud.k2view.com>
3. If you plan to install TDM, you will need access to K2view’s Exchange

4.2. Whitelisting

Certain functions require HTTPS access over port 443:

- **K2view Agent** – makes outbound REST calls to <https://cloud.k2view.com> to obtain deployment and configuration instructions via the K2cloud Orchestrator's Mailbox service
- **For Installations and Updates** –
 - <https://share.cloud.k2view.com> to fetch K2view Fabric, Web Studio and k2-agent images
 - <https://github.com> to pull K2view blueprints
 - <https://github.com> to pull K2view deployment Helm charts

You may need to whitelist data sources/targets and open HTTPS ports to meet your needs.

4.3. DNS

Using [Context-Based URLs](#) for your spaces allows you to define a single DNS A record. This is available with Fabric 8.2 and later. Before Fabric 8.2, a wildcard certificate and domain needed to be provisioned.

The Terraform configuration creates an Azure DNS Zone by default. It will configure everything you need to get up and running.

If you use a preexisting DNS zone, you need this domain name to create your Ingress Controller and register a DNS A record. You must create a DNS record pointing to the node hosting the K8s Ingress Controller (it can point to a private or a public IP). The DNS record for the server hosting the K8s node does not need to be registered in the public DNS. The only requirement is that users can resolve the DNS name internally to the customer's environment.

4.4. TLS keys and certificates

To use K2view's Azure Blueprint and Terraform, you need a TLS PEM certificate and its associated TLS PEM Key file.

Your network team must create a TLS certificate for the domain you selected as your Domain Ingress. The entire certificate authority chain needs to be present. During the installation, you must specify the path to your PEM TLS certificate and its associated private key.

Please note that K2view can provide both domain and certificates for a Proof of Technology (POT) environment if you find this useful.

4.5. Permissions and Roles

Permissions and roles are required by the user performing the installation.

- **Azure Resource Registration.** A **Subscription Owner** must register Azure resources before installation. Here is the list of resources:
 - **Microsoft.ContainerService:** Manages AKS resources.
 - **Microsoft.Network:** Supports virtual networks and related resources.
 - **Microsoft.Compute:** Required for provisioning virtual machines.
 - **Microsoft.Storage:** Handles storage resources used by AKS.

To register these resource providers, you can use the following Azure CLI:

```
az provider register --namespace Microsoft.ContainerService
az provider register --namespace Microsoft.Network
az provider register --namespace Microsoft.Compute
az provider register --namespace Microsoft.Storage
```

4.6. Persistent Storage

When configuring your Kubernetes nodes, K2view needs the [“Zone Redundant Storage”](#) type of storage to configure your environment.

- If you are using an existing cluster, we need to install the relevant CSI drivers
- If you use K2view’s [Azure Blueprint](#), it comes with ZRS Support

For persistent storage in an Azure Kubernetes Service (AKS) environment, we recommend using **Azure Premium Files with Zone-Redundant Storage (ZRS)**. ZRS ensures high availability and durability by replicating data synchronously across three availability zones within the same region, providing resilience against zone failures. Unlike Azure Managed Disks, which are confined to a single availability zone, ZRS allows pods to access shared storage seamlessly, even if rescheduled to different zones. This makes ZRS an ideal choice for workloads requiring high availability and shared access, such as stateful applications or shared configuration files in distributed environments.

Using Azure Premium Files with Zone-Redundant Storage (ZRS) is a recommendation, not a strict requirement. Customers can use Azure Files (standard) with LRS, particularly for non-production instances where performance and resilience may not be as critical.

However, there are some important considerations to be aware of when using LRS - particularly for production usage:

- LRS (Locally Redundant Storage) is tied to a specific availability zone (AZ). This means the storage volume is accessible only within the same AZ where it was provisioned.

- In Kubernetes environments (AKS), if a pod using this volume is rescheduled to a node in a different availability zone, the pod will fail to mount the volume. This will trigger retry attempts until the pod is rescheduled to a node in the same AZ as the storage. This behavior can introduce delays or disruptions in scenarios where pods must be rescheduled frequently (e.g., node failures, cluster scaling events, maintenance windows, etc.).

While LRS can work for non-production environments, customers should know the potential for cross-AZ rescheduling failures. For production environments, ZRS remains the recommended option because it avoids this issue by providing zone-resilient storage that is accessible across all zones in the region.

4.7. Tools

The following tools are required to perform the installation by the user with the required [Permissions and Roles](#).

Please install the following tools:

- Terraform – version 1.9.x - [HashiCorp install guide](#)
- Kubectl – version 1.14.0 - [Kubectl install guide](#)
- Azure CLI - [Azure CLI install guide](#)
- Helm – version 2.13.0 - [Helm install guide](#)
- Docker – latest version - You can use the tools provided by your container registry to pull from the K2view container registry and push to your registry. Docker or an OCI-compatible tool will likely be needed for this.

4.8. Git Code Repository

A Git code repository is required for your Fabric Project. Three pieces of information will be required to create your K2view project including:

- Repository URL
- Git token
- Git Branch / Tag

4.9. Container Registry

K2view will create a container registry account, user accounts, and associated configurations for you. You will use the K2view container registry account to obtain K2view Docker images. In Step 5, K2view will require the location of the images you push to your Container Registry.

By default the [Terraform's variables file](#) is configured to create an Azure Container Registry for you. If you desire to use a pre-existing Container Registry you may do so. Whether you use the one created by K2view or an OCI-compatible container registry, the location where you will be populating K2view Docker images must be provided to K2view.

You must populate your selected Container Registry by pulling and pushing docker images. Docker must be installed on a computer to pull and push docker images from the K2view container registry to the

Container created. You will need a K2view container registry account you can obtain from your K2view representative

Outbound HTTPS/443 Access

You need to confirm outbound HTTPS access over port 443 for:

- The host “pulling” from K2view’s container registry has outbound HTTPS access over port 443.
- Users have HTTPS access to the K2view Cloud Manager over port 443.

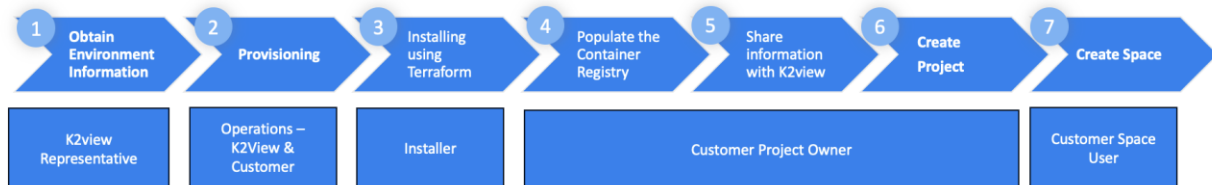
Extracting and pushing Docker images to your Container Registry

- You will need the K2view container registry account to perform the “pull” action. This will be provided to you by your K2view representative.
- You will need access to your Container Registry to publish K2view and other Docker images.
- You must note the location of the required Docker Images posted to your Container Registry. K2view needs this information to complete your provisioning in Step 5.

5. Installation Checklist Worksheet

Use this section to gather the information you need before installing. For more information, please consult the [Prerequisites](#) section.

At various steps during the installation, you must obtain information from K2view and, in Step 5, provided information to K2view.



The following summarizes the information you and K2view require.

Assets you will need for your installation

As input to your [Terraform-based installation](#), you will need to have obtained:

- [] **Domain Ingress FQDN:** _____
Subdomain of your cluster. A DNS A record of site.domain.com will point to the load balancer IP.
- [] **TLS PEM certificate file:** A file
A full CA chain certificate for the Domain Ingress (e.g., fullchain.pem of site.domain.com)
- [] **Associated TLS PEM Private Key:** A file

You have confirmed Internet connectivity to specific locations

Certain functions require HTTPS access:

- [] **K2view Agent** – makes outbound REST calls to <https://cloud.k2view.com> (port 443) to obtain deployment and configuration instructions via the K2cloud Orchestrator's Mailbox service.
- [] **<https://share.cloud.k2view.com>** (port 443) to fetch K2view Fabric, Web Studio and k2-agent images.
- [] **<https://github.com>** (port 443) to pull K2view blueprints and deployment Helm charts.
- [] You may need to whitelist specific data sources/targets and open HTTPS ports as required to meet your needs.

Please share with K2view – Required for Step 2

Please share this with your K2view representative for this information.

[] K2view Admin user:

○ Name: _____

○ Email address: _____

○ Phone number with country code included: _____

Please request a K2view container registry Account – Required for Step 3

Please request that your K2view representative provide you with an account for the **K2view** container registry. This account is a shared account intended for your entire organization.

Shared by K2view – Required for Step 3

Once the above is shared with your K2view representative you will be provided the following information.

[] K2view Cloud Mailbox ID provided to you by K2view: _____

[] K2view container registry Account supplied by K2view: _____

[] List of Docker images to pull from the container registry:

Recommended by K2view. Please consult with your K2view representative for this information.

After completing Step 4—the image population of your Container Registry—and before creating your K2view Project and your first Space, you must provide the following to K2view.

- [] List of Docker images you pushed to your Azure Container Registry:

Please share this information with your K2view representative. After you have populated your container registry, this list of locations must be shared with your representative K2view.

- [] Domain Name of your Ingress Controller:

It can be shared with K2view before the installation if you have selected the domain

- [] Cloud Provider:

This information will help K2view better understand your deployment.

- [] A site name and purpose:

This optional information such as POT, DEV, NPRD, PROD will be used by K2view to differentiate your first site better from others you might create in the future

K2view Project Creation Step – Required for Step 6

You will need the following information to create your project in Step 6.

[] Git Repository URL: _____

[] Git token (do not share): _____

[] Git Branch / Tag: _____

6. Instructions Summary

6.1. Cloning the K2view Blueprints

After installing a Git client on your machine, you must “clone” the K2view Blueprints. The blueprints incorporate the Fabric Docker Compose Runtime blueprint.

Select a directory to host the K2view Blueprints (Internet access required). Using a shell, change to your Git directory and run the following command:

```
git clone https://github.com/k2view/blueprints.git
```

6.2. Configuring Your Deployment

Various configurable settings are available when using K2view’s [Azure Blueprint](#) and Terraform. Some of these are noted here. Please review the Terraform variables file to see a full list of Azure Kubernetes Services (AKS) configurable values. K2view can provide you with assistance in configuring your environment.

Notable settings include the following:

- Azure Resource Group. The selected resource group. The user needs permission to create resources within this group.
- Region. The Azure Region of the cluster.
- Zones. The list of availability zones for your deployment.
- Azure Instance Type: The instance types (a VM SKU) associated with the AKS Node Group. You might consider working with your K2view representative or looking at K2view’s [Kubernetes requirements](#) for guidance.
- Domain. The domain name of the Ingress Controller.
- Cluster Name. The name of your cluster.
- Min/Max/Desired Workers. Values associated with worker nodes you intend to employ for auto-scaling.
- Azure Container Registry Creation. Whether to create the ACR. True by default.
- DNS Zone Creation. Whether to create a DNS Zone. True by default.
- Ingress Creation. Whether to deploy and Nginx Ingress Controller. True by default.
- Mailbox ID. The Mailbox ID will be provided to you by K2view.

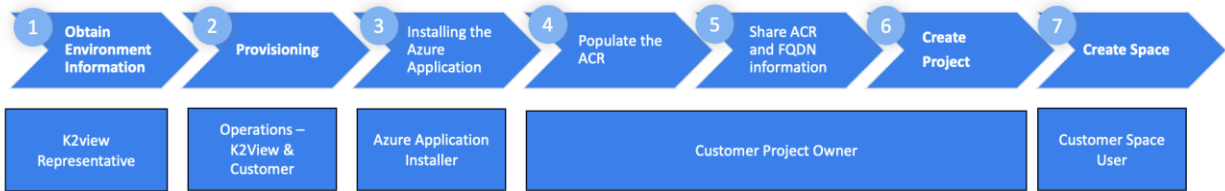
Network Configuration

The Azure configurable values of the [Terraform variables file](#) can also be used to configure various settings. Default values are provided. If you have specific needs, please review this configuration file where you will find these.

- Vnet and subnet configuration: CIDR range of the Vnet and subnets
- NAT Gateways creation

Other parameters are specified and should be consulted.

6.3. Populating Your Azure Container Registry



To populate your Azure Container Repository, you will need

1. K2view container registry Account supplied to you by K2view
2. Obtained the list of Docker images to pull from container registry and push to in consultation with K2view
3. Docker installed on the computer used to pull and push Docker images to your ACR

The [Planning](#) section describes what is necessary to be ready to perform this step.

6.3.1. Commands to Pull Containers from the K2view container registry

The list of K2view service Docker images will be discussed and recommended to you in consultation with your K2view representative.

For these instructions we will “pull” from the K2view container registry three images of a specific version: fabric-studio, fabric, and the k2-cloud-deployer Docker images.

Docker tools need to be installed on the computer you are performing this installation. You can download these from <https://docker.com>.

First Login

Command:

```
docker login -u [your K2view container registry username]
https://docker.share.cloud.k2view.com
```

Second, pull the required images. In this example, we pull three.

Commands: In succession, please run the following three commands. Keep in mind that the version and docker images needed may differ.

```
docker pull docker.share.cloud.k2view.com/k2view-images/fabric-studio:8.3.0_291
docker pull docker.share.cloud.k2view.com/k2view-images/fabric:8.3.0_291
docker pull docker.share.cloud.k2view.com/k2view/k2-cloud-deployer:1.8.14
```

6.3.2. Commands to Push Containers to the Azure Container Registry

Now that you have “pulled” images from the K2view container registry, you are now ready to “push” these to the Azure Container Registry (ACR) created for you. If you wish to use a preexisting ACR or OCI-compliant container registry, please also consult the [Selecting a Different Azure Container Registry or OCI-Compliant Registry](#) section.

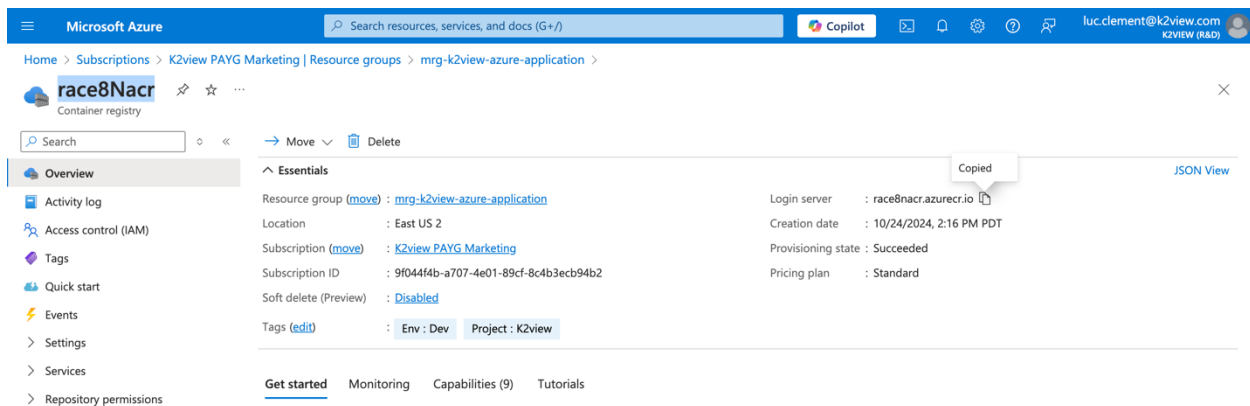
First, obtain the credentials required to perform a Docker Push to your ACR

The AKS Terraform can be configured to create an ACR. You can obtain the credential associated with it by navigating you the resource group and selecting the ACR created for you.

In this example, the Managed Application Resource Group is named mgr-k2view-azure-application, and the created ACR is named rac8Nacr (a random name).

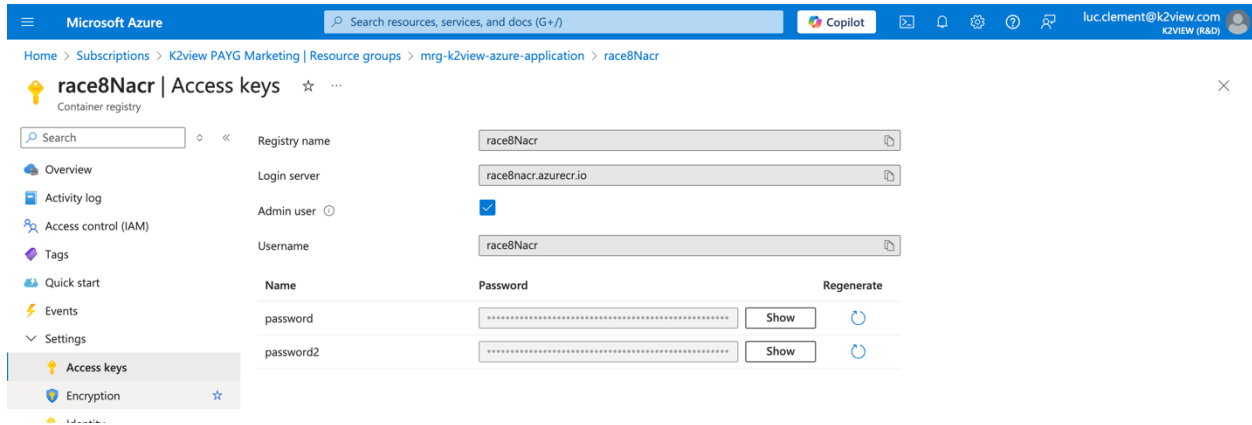
When you navigate to this ACR location, please

1. Get the URL of the login server, which is shown here. Copy the URL which will be required for your Docker login and push commands.



2. Select Access Keys in the left navigation menu under Settings to obtain your credentials. From this page:
 - a. Copy the username
 - b. Use the Show button to obtain the password, and copy this password

This information will be required to perform the login to your ACR using a Docker command



Second, perform the Docker Push to your ACR

Use the Docker login command to login to your ACR

```
docker login -u [your ACR username] [the ACR Login server]
```

In our example we used: `docker login -u race8Nacr race8nacr.azurecr.io`

Third, tag each image you pulled from the K2view container registry

Use of tags is for version control. Tagging Docker images allows you to maintain different versions of the same application, making it easier to roll back to a previous version if needed. This can be especially useful in cases where a new release introduces a bug or breaks compatibility with other components.

In our example, we used the following three commands, respectively, for the fabric-studio, fabric, and k2-cloud-deployer Docker images. Note that the [registry] value is the name used in our example; you will need to use the one associated with your container registry.

```
docker tag docker.share.cloud.k2view.com/k2view-images/fabric-studio:8.3.0_291
[registry]/k2view-images/fabric-studio:8.3.0_291
```

```
docker tag docker.share.cloud.k2view.com/k2view-images/fabric:8.3.0_291
[registry]/k2view-images/fabric:8.3.0_291
```

```
docker tag docker.share.cloud.k2view.com/k2view/k2-cloud-deployer:1.8.14
[registry]/k2view/k2-cloud-deployer:1.8.14
```

Fourth, push images to your ACR

Next, you need to push the Docker images you pulled from the K2view container registry to your container registry using the command:

```
docker push [image location]
```

In our example, we used:

```
docker push [registry]/k2view-images/fabric-studio:8.3.0_291
```

```
docker push [registry]/k2view-images/fabric:8.3.0_291
```

```
docker push [registry]/k2view/k2-cloud-deployer:1.8.14
```

Fifth, please provide the locations you used to K2view

K2view needs these ACR locations of your images to finalize your site's configuration. Please note that you will share these locations with K2view as described in the [Installation Checklist Worksheet](#) section.

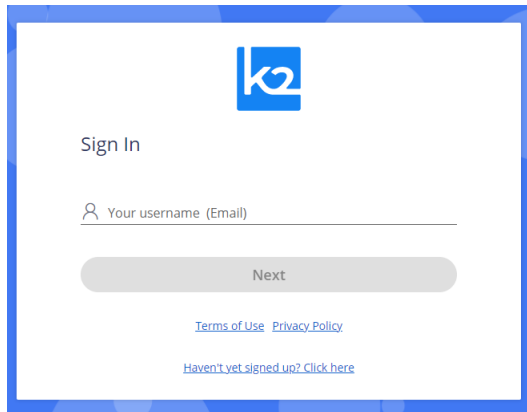
6.3.3. Selecting a Different Azure Container Registry or OCI-Compliant Registry

You must follow similar steps if you choose to use a pre-existing ACR or an OCI-compliant registry. You will also need to provide K2view with the locations of your registry.

6.4. Sign In to K2cloud

To sign in, access cloud.k2view.com using Chrome, Microsoft Edge, or Firefox web browsers.

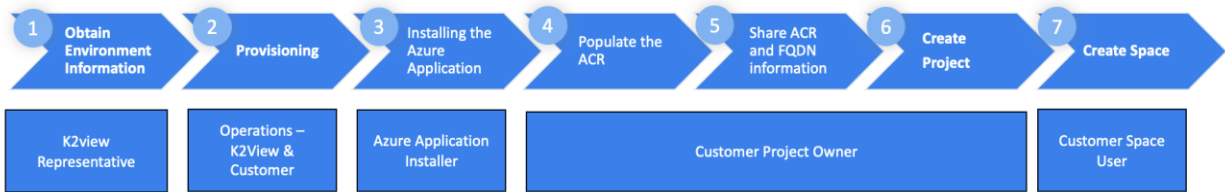
When prompted to sign in, enter your email address. This account needs to have been provisioned for you by K2view. Please refer to the [Prerequisites](#) section.



If provisioned by K2view, you will receive an access code via email you will then enter in the prompt window that is shown when you press Next.

You will be directed to the Spaces page, where you will eventually create spaces. First, you need to create a project.

6.5. Create the Project



You create a project by selecting the Projects top-level navigation tab and pressing “Create Project”. A project requires a name and an associated Git repository. You must provide Git-related information to create a project. To learn more about Git and how to obtain the Repository URL you will use, a branch, and your token, please review the [GitHub](#) section.

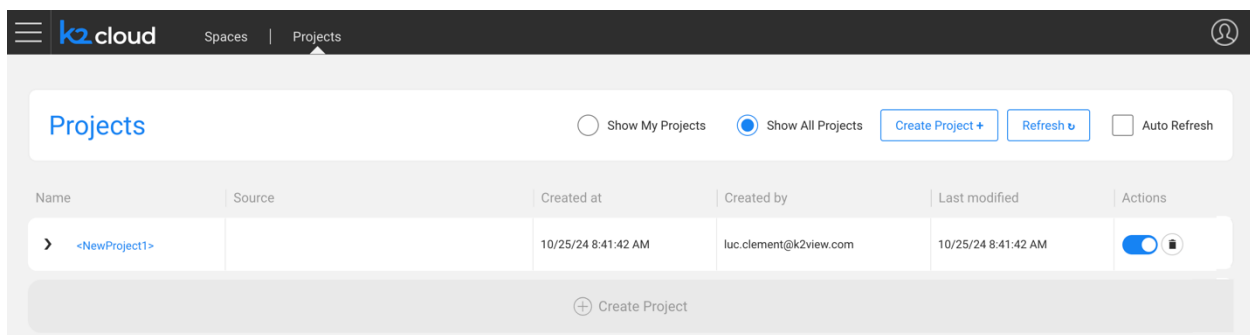
The Fabric spaces you implement are based on a “Space Profile”. You can configure more than one Space Profile per Project. If you need assistance, a K2view representative will assist you. Some of this information will have been shared with you during the [Planning](#) phase.

To create a Space Profile, you need to:

- Give it a name that will have significance to your users
- Select an Image Profile identified in consultation with K2view, and
- Define the site where this space will operate within. K2view will help you define this list.

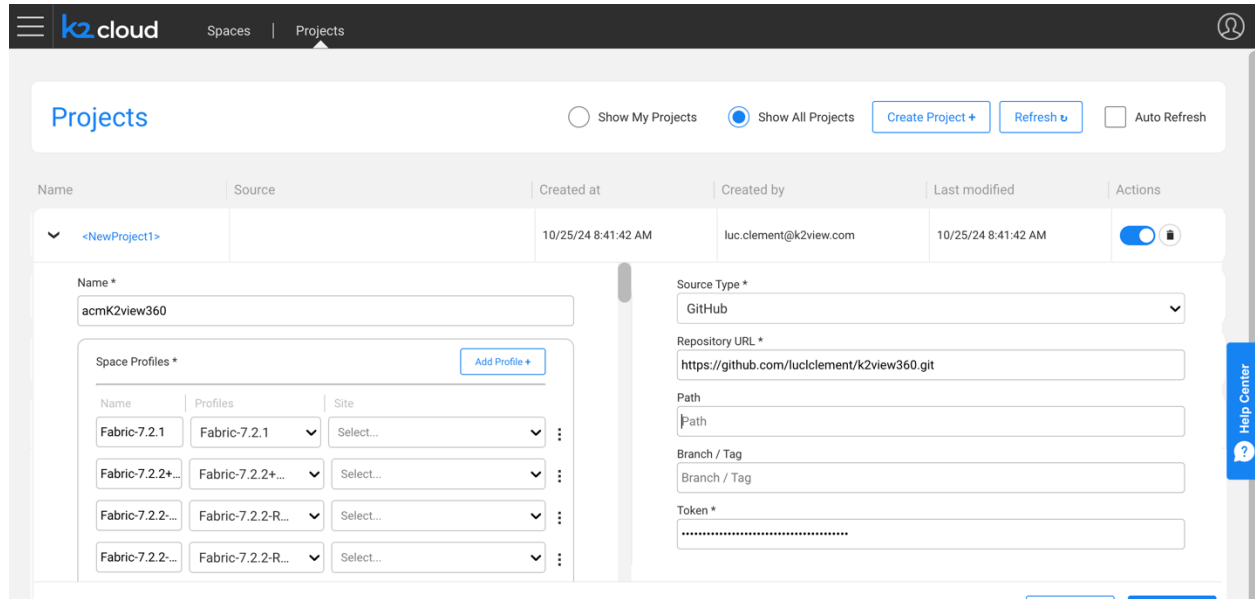
Creating a Project

After pressing Create Project, an empty project will be created.



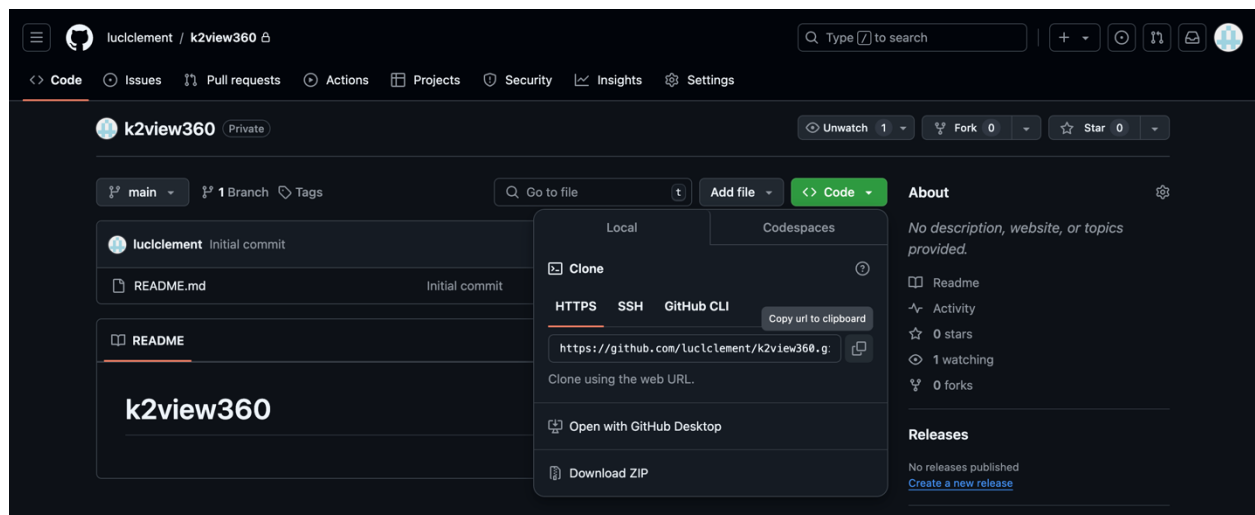
Provide the project with a name and select your space profiles. Again, K2view will have provided this.

You also need to provide the details of your Git source control system.

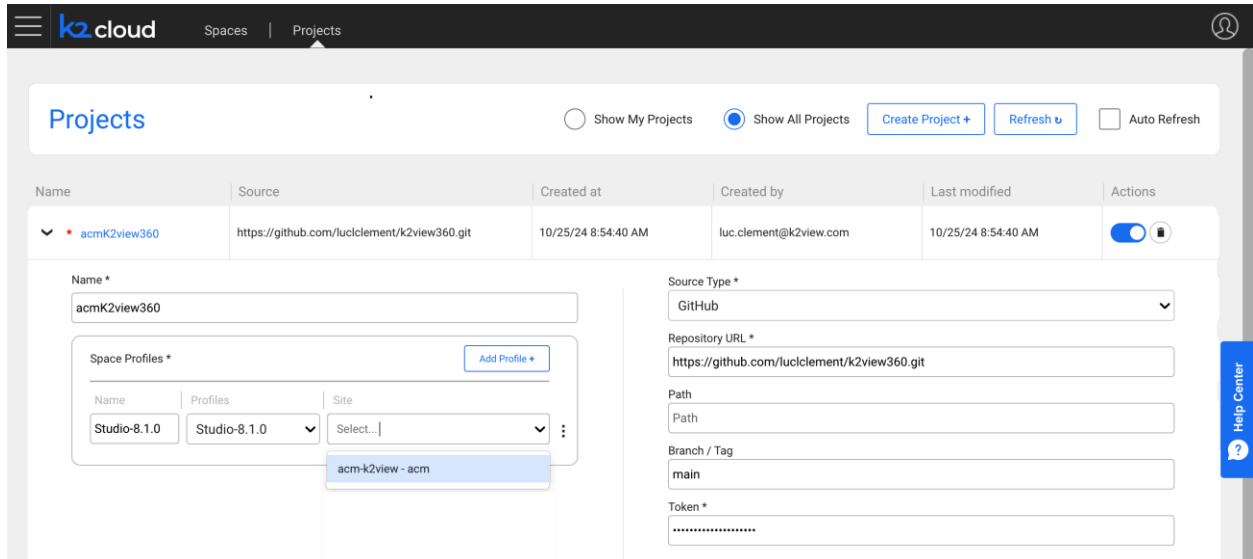


The screenshot shows the 'Projects' page in the K2view interface. At the top, there are tabs for 'Spaces' and 'Projects', with 'Projects' selected. Below the tabs, there's a 'Projects' header with options to 'Show My Projects', 'Show All Projects' (selected), 'Create Project +', 'Refresh', and 'Auto Refresh'. A table lists projects, with the first one being '<NewProject1>'. Below the table, the 'Create Project' form is visible. It includes fields for 'Name *' (set to 'acmK2view360'), 'Source Type *' (set to 'GitHub'), 'Repository URL *' (set to 'https://github.com/luccllement/k2view360.git'), 'Path', 'Branch / Tag', and 'Token *'. There's also a section for 'Space Profiles *' with a table of profiles and a 'Add Profile +' button.

For example, you can obtain the URL from your Git repository, as shown here in our example.



And, finally, please select the site created from the site's dropdown.



The screenshot shows the K2view cloud interface. At the top, there's a navigation bar with 'Spaces' and 'Projects' tabs. The 'Projects' tab is active, showing a table of projects. Below the table, there's a form to create a new project. The form includes fields for Name, Source Type, Repository URL, Path, Branch / Tag, and Token. A dropdown menu for 'Site' is open, showing a list of sites, with 'acm-k2view - acm' selected.

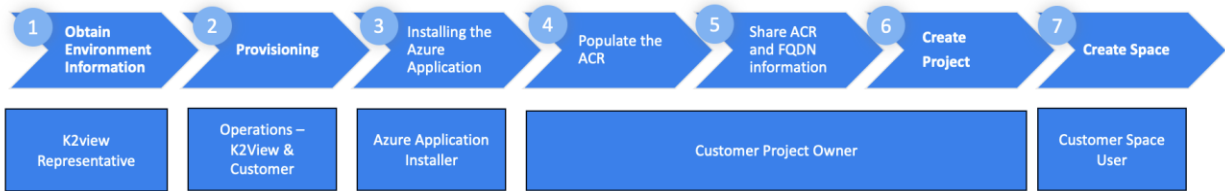
Name	Source	Created at	Created by	Last modified	Actions
acmK2view360	https://github.com/luccllement/k2view360.git	10/25/24 8:54:40 AM	luc.clement@k2view.com	10/25/24 8:54:40 AM	

Form Fields:

- Name *: acmK2view360
- Source Type *: GitHub
- Repository URL *: https://github.com/luccllement/k2view360.git
- Path: Path
- Branch / Tag: main
- Token *:
- Space Profiles *:

Name	Profiles	Site
Studio-8.1.0	Studio-8.1.0	Select...

6.6. Creating Your First Fabric Space



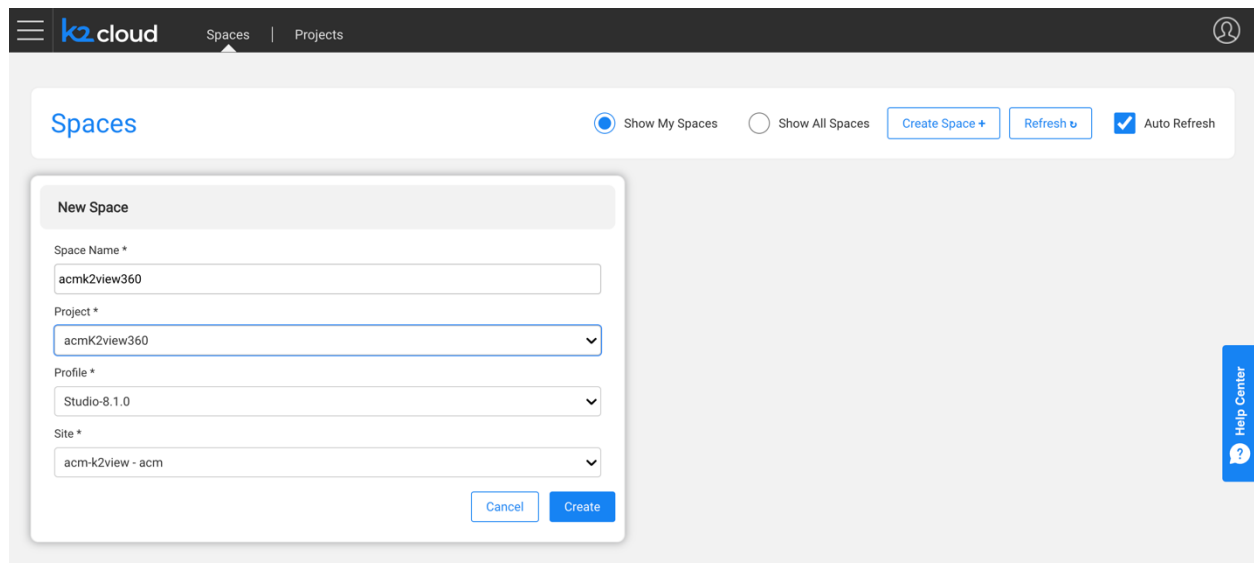
After your Project is created, you can create a Fabric Space. Here, we show an example of an existing space and a new space being defined.

To create a space

- Give it a name,
- Select the project you created,
- Select your intended profile, one recommended by K2view given your intended purpose, and
- Select site.

If you need to install a K2view extension such as TDM, you must go to your space and install the extension from K2exchange.

When ready, press Create to trigger the space's creation.

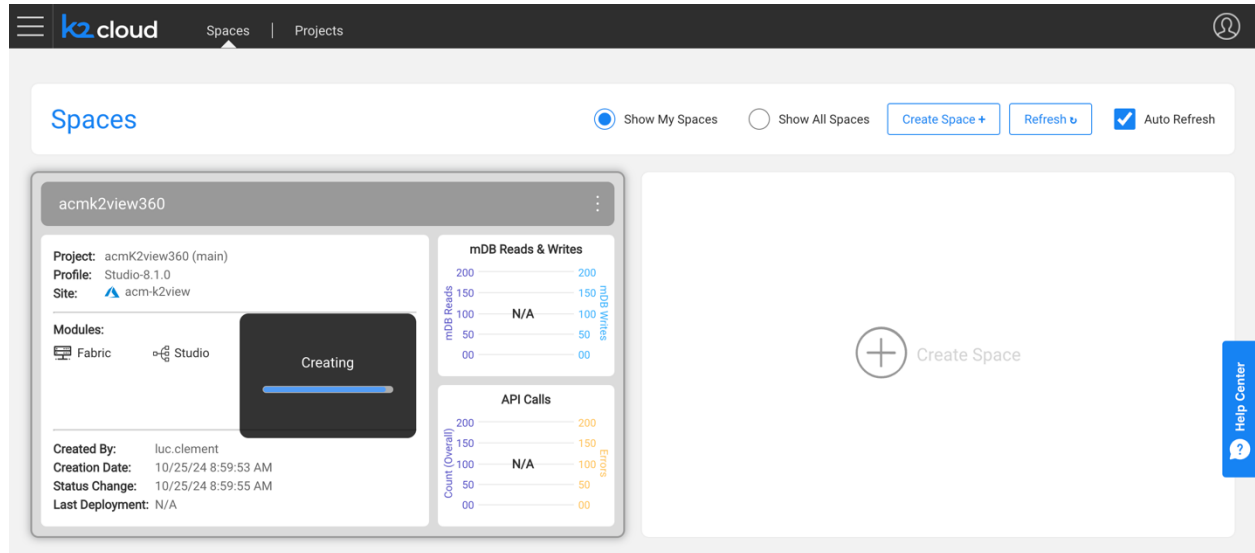


The screenshot shows the K2cloud interface. At the top, there's a navigation bar with 'Spaces' and 'Projects' tabs. Below the navigation bar, there's a 'Spaces' section with a filter for 'Show My Spaces' and a 'Create Space +' button. A 'New Space' modal is open, showing the following fields:

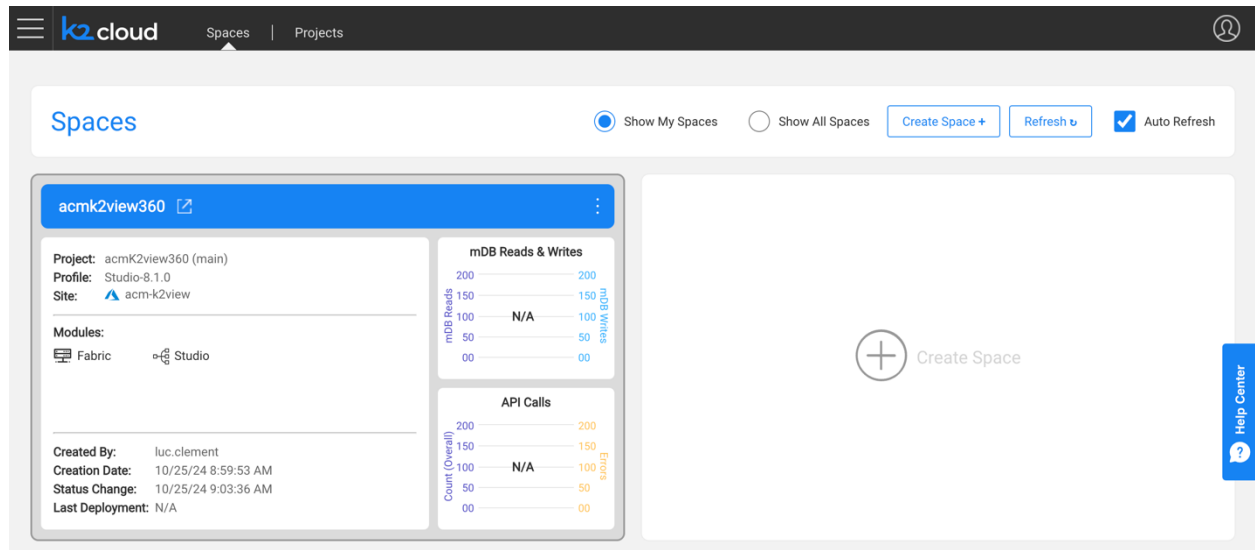
- Space Name *: acmk2view360
- Project *: acmk2view360
- Profile *: Studio-8.1.0
- Site *: acm-k2view - acm

At the bottom of the modal, there are 'Cancel' and 'Create' buttons. On the right side of the page, there is a 'Help Center' button.

The Space creation process begins and may take a few minutes to complete.



Once complete your site is displayed to you.



You have successfully created your Project and Space and are ready to begin working with K2view.

7. Context-Based URLs

Fabric 8.2 introduced a new option for accessing Fabric Spaces. This feature, called Context-Based URLs, allows Fabric Spaces to share the same domain and be accessed via context-based paths (e.g., <https://acm.domain.com/acmk2360ne-k2v/app/studio/>). This feature eliminates the need for separate subdomains and allows a single TLS certificate to cover all spaces, simplifying the certificate management process. This enhancement reduces setup and maintenance complexities, offering users a more streamlined approach to securing and accessing Fabric spaces. To configure this capability with newly created spaces, set the `PATH_BASE_ROUTING` parameter to `'true'` in the `config.ini` file.

A path to Web Studio using a subdomain-based Fabric Space URL type previously would have been

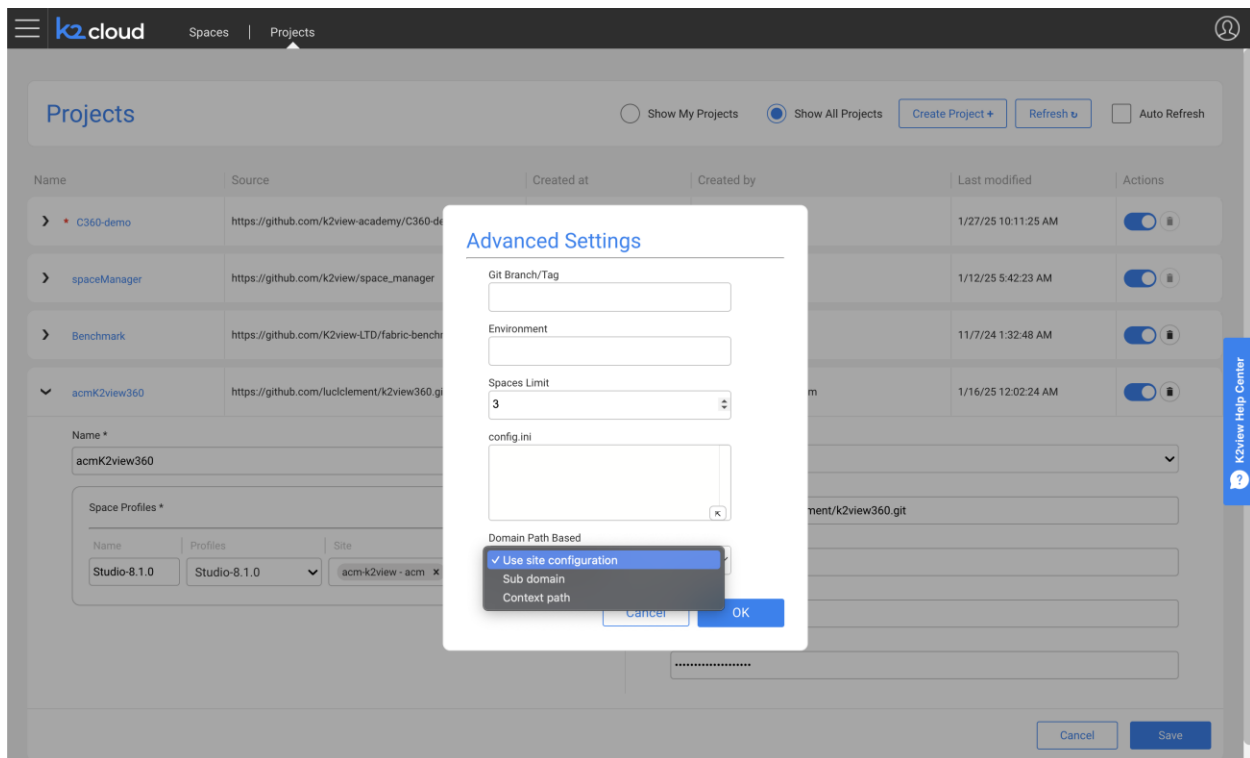
<https://acmk2360ne-k2v.acm.domain.com/app/studio/>.

Context-based URLs make it possible to create a Space using a URL Context Fabric Space accessible at:

<https://acm.domain.com/acmk2360ne/k2v/app/studio/>

Making use of the Capability

This feature can be exercised when a site is created. It is enabled using the Advanced Settings dialog of the space profile associated with the site.



Feature Summary: Subdomains or URL Contexts for Fabric Space URLs

This feature introduces flexibility in accessing Fabric Spaces by allowing one of two types of URLs:

1. **Subdomain-Based URLs (existing method):** Each Space requires a dedicated subdomain (e.g., <https://acmk2360ne-k2v.acm.domain.com/app/studio/>) and a wildcard TLS certificate.
2. **Context-Based URLs (new method):** Spaces can now share the same domain, accessed via context-based paths (e.g., <https://acm.domain.com/acmk2360ne-k2v/app/studio/>). This eliminates the need for separate subdomains and allows a single TLS certificate to cover all Spaces, simplifying certificate management.

This enhancement reduces complexity in setup and maintenance, offering users a more streamlined approach to securing and accessing Fabric Spaces.